RESCUEBENCH: EVALUATING LARGE LANGUAGE MODELS ON DYNAMIC SPATIAL REASONING AND CONSTRAINT SATISFACTION

Akshat Bhat, Trusha Talati, Akhil Gogineni & Nicolas Yepes
Department of Computer Science
University of Illinois Urbana-Champaign
Champaign, IL 61820, USA
{akshatb5,ttalati2,vg23,nyepes2}@illinois.edu

ABSTRACT

Large Language Models (LLMs) struggle to execute plans within strict, deterministic mathematical boundaries. We introduce RescueBench, a novel benchmark designed to evaluate LLM-based agents on dynamic spatial reasoning and physical constraint satisfaction within the domain of Humanitarian Logistics. RescueBench comprises 20 static JSON environments that model a 16-node city graph with a heterogeneous, 4-class vehicle fleet. It tests an agent's ability to transition from parsing unstructured semantic emergency feeds to solving the Capacitated Vehicle Routing Problem with Time Windows (CVRP-TW). The benchmark scales across four difficulty tiers, culminating in dynamic scenarios where hidden infrastructure failures alter the graph mid-mission. We provide an objective, Python-based evaluation protocol that utilizes Dijkstra's algorithm to penalize constraint violations and measure routing efficiency. By releasing RescueBench, we provide the research community with a strictly quantifiable environment to test the limits of agentic self-correction.

1 INTRODUCTION

Large Language Models (LLMs) frequently fail at tasks requiring strict spatial reasoning and multi-step constraint satisfaction. While models generate plausible natural language plans, they often hallucinate physically impossible actions, such as routing vehicles through blocked intersections or exceeding physical weight limits (Song et al., 2026). Testing an agent's ability to adhere to these physical laws requires an environment bounded by deterministic mathematics, rather than semantic text evaluation.

Existing agent benchmarks do not adequately capture the complexities of Humanitarian Logistics and Emergency Resource Allocation. This specific domain forces a continuous transition between unstructured semantic reasoning (parsing text-based emergency feeds) and rigorous spatial optimization (updating a mental graph and calculating constraints). Current benchmarks lack the strict operations research mechanics required to test pure constraint satisfaction. Furthermore, they are primarily static; they do not test an agent's ability to abandon a plan and recalculate routes when the environment fundamentally changes mid-execution.

We introduce RescueBench to bridge this gap. RescueBench is a dataset of 20 deterministic JSON environments that evaluate an agent's capacity for spatial routing and dynamic replanning. The benchmark models a city as a 16-node, 20-edge directed graph. Agents must dispatch a 4-class heterogeneous fleet (ambulances, fire engines, police cars, and supply trucks) to resolve time-sensitive incidents. The environments enforce strict physical constraints, including vehicle capacities, varying travel speeds, and edge-weight limits that prohibit heavy vehicles from crossing specific bridges.

RescueBench evaluates agents across four scaling difficulty tiers. Tier 1 tests basic single-unit routing. Tier 2 introduces mathematical capacity constraints. Tier 3 forces ethical prioritization under forced resource scarcity. Tier 4 introduces hidden dynamic triggers, where mid-mission events—such as flash floods—destroy edges and force immediate agent replanning. By utilizing

an objective Python evaluation layer rooted in Dijkstra's algorithm, RescueBench offers a highly reproducible, mathematically rigorous environment to evaluate the limits of LLM planning capabilities.

2 Related Work

Existing benchmarks for LLM reasoning primarily evaluate static logic rather than dynamic spatial routing. Benchmarks such as GSM8K evaluate mathematical problem-solving through text, while BoardgameQA evaluates static state-tracking. While the ReAct framework (Yao et al., 2023) demonstrated that interleaving reasoning and acting improves performance in text-based semantic environments, it struggles in deterministic mathematical spaces. RescueBench isolates this failure point by requiring agents to continuously update an internal mathematical state model as they move resources across a physical graph over a shifting temporal clock.

Sequential decision-making benchmarks often lack operations research constraints. Environments like ALFWorld evaluate an agent's ability to follow text-based instructions in simulated households. While these benchmarks test tool use, they do not penalize suboptimal travel efficiency or require combinatorial resource matching. Recent studies highlight that end-to-end LLM solvers frequently hallucinate mathematics and output infeasible routes when applied to complex routing problems (Zhang et al., 2025; Li et al., 2025). RescueBench addresses this by utilizing Operations Research standards, specifically the Capacitated Vehicle Routing Problem with Time Windows (CVRP-TW), as its core evaluation engine.

Prior work establishes that LLMs must act as heuristic orchestrators rather than direct mathematical solvers in mission-critical environments (Wang et al., 2025). RescueBench provides the necessary interactive simulation environment to test this hypothesis, forcing autonomous LLM agents to engage in real-time self-correction loops when their natural language plans violate strict physical constraints. Prior datasets in the emergency response domain typically provide historical dispatch logs to train models for demand forecasting. RescueBench, conversely, tests real-time reactive reasoning.

3 Benchmark Design

3.1 Task Domain and Scope

RescueBench operates strictly within Humanitarian Logistics and Emergency Resource Allocation. This domain forces a continuous transition between unstructured semantic reasoning (parsing emergency feeds and dynamic triggers) and rigorous spatial optimization (updating a mental graph and solving the VRPTW). The benchmark evaluates physical constraint satisfaction (e.g., respecting bridge weight limits) and dynamic adaptability (e.g., rerouting when a road washes out). It is out of scope to test natural language dialogue generation or visual processing. RescueBench is a wholly original framework designed specifically around CVRP-TW mechanics.

3.2 Task Descriptions

RescueBench consists of 20 modular JSON files. Each file contains a fully initialized state including a graph (`nodes` and `edges`), an `active_fleet`, a list of `incidents`, and a list of `dynamic_triggers`.

Example Task 1: Capacity Mathematics (Tier 2, Scenario 7). In this static scenario, the agent receives an input state detailing a city bus rollover requiring a medical capacity of 4. Figure 1 illustrates the full topology of this scenario, displaying the incident location (`node_int_10`) relative to the available fleet. Because the available ambulances have a strict maximum capacity of 2, the expected output behavior requires the agent to dispatch two separate ambulances to the same incident node. The success criteria mandate that both vehicles arrive before the global clock exceeds the deadline minute. This task is challenging for standard LLMs because it requires mathematical deduction across the entire graph; the agent must recognize that a single dispatch is mathematically insufficient and coordinate multiple routing paths simultaneously.

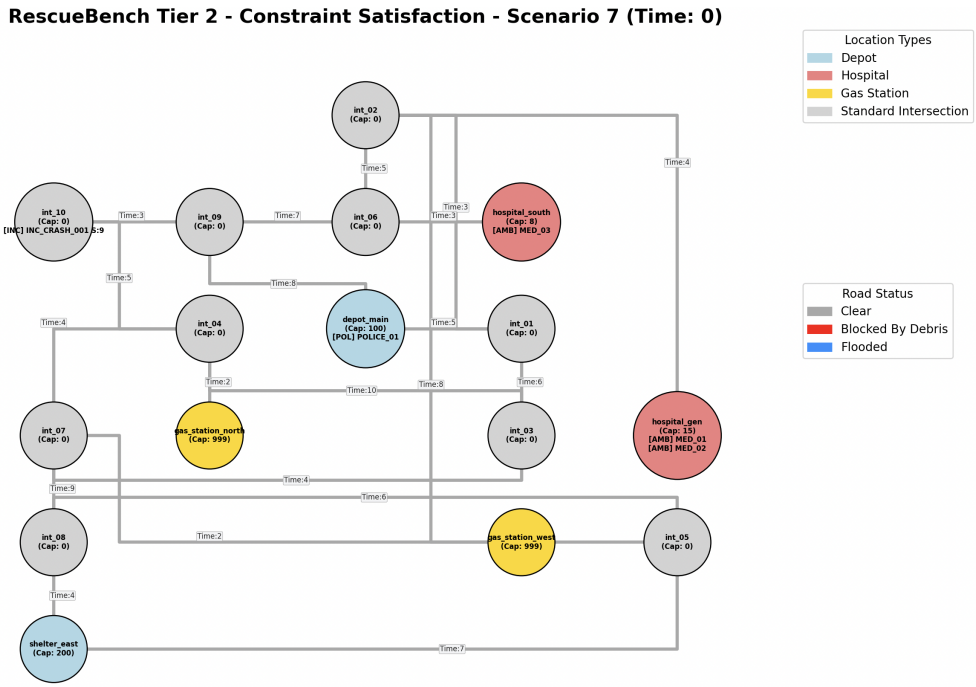

Figure 1: Visualization of Example Task 1 (Tier 2, Scenario 7). The full city graph displays the incident at `int_10` alongside the distributed ambulance fleet at `hospital_gen` and `hospital_south`. The agent must parse this topology and mathematically deduce that multiple ambulances are required to meet the incident's capacity constraint.

Example Task 2: Immediate En-Route Diversion (Tier 4, Scenario 16). In this dynamic scenario, the agent initially routes an ambulance across a suspension bridge to reach a critical patient. At minute 8 of the simulation, a hidden trigger fires, altering the JSON state to mark the bridge as `blocked_by_debris`. The Python environment injects a natural language warning into the agent's context (see Figure 2 in the Appendix). The expected behavior is for the agent to discard its previous path, re-query the graph state, and route the ambulance via a longer southern detour. The success criteria require the agent to avoid attempting to cross the destroyed edge. This is challenging because it breaks single-shot planning architectures and necessitates a robust self-correction loop.

3.3 Task Design Principles

The design of RescueBench is guided by the principle of deterministic verifiability. Every constraint is Boolean or algebraic, ensuring that success is indisputable. We ensured diversity by varying the required capabilities across four distinct emergency themes: medical (ambulances), structural (fire engines), security (police), and logistics (supply trucks). Difficulty was calibrated iteratively against a deterministic Dijkstra baseline; if a mathematical path to victory did not exist, the task was redesigned. We calibrated the upper bounds of difficulty against a zero-shot LLM baseline, tightening deadlines until single-shot planning consistently failed.

3.4 Difficulty Distribution

The 20 benchmark tasks are evenly distributed across four difficulty tiers. Table 1 summarizes this distribution. The columns denote the tier level, the primary reasoning constraint evaluated by the tasks within that tier, and a description of the core trap. The distribution scales linearly from easy (Tier 1) to expert (Tier 4).

Table 1: Distribution of the 20 RescueBench tasks across the four difficulty tiers. The table illustrates the scaling complexity of the reasoning constraints evaluated in each tier.

Tier	Tasks	Primary Constraint Evaluated
1: Basic Triage	1–5	**Direct Routing:** Single-unit dispatches on clear roads. Evaluates basic graph comprehension.
2: Constraint Math	6–10	**Physical Limits:** Evaluates adherence to edge weight limits, fuel capacities, and combinatorial capacity math.
3: Ethical Prioritization	11–15	**Forced Scarcity:** Evaluates the ability to maximize the Priority-Weighted Resolution Score by sacrificing low-severity incidents.
4: Dynamic Replanning	16–20	**Mid-Mission Surprises:** Evaluates self-correction loops when hidden triggers alter graph topology mid-execution.

4 Evaluation Protocol

Agents interacting with RescueBench are evaluated using strict Operations Research metrics. Because the environments are deterministic JSON files, evaluation is handled entirely by an objective Python scoring script, eliminating subjective human grading.

Metrics and Scoring: We define three core metrics. 1. *Priority-Weighted Resolution Score (PWRS):* Calculated as $S = \sum(w_i \times I_i)$, where w_i is the incident severity weight (1-10) and I_i is a binary indicator of whether the incident was resolved before its deadline. 2. *Response Efficiency:* Calculated as $E = \frac{T_{optimal}}{T_{agent}}$, comparing the agent's total travel time against the mathematical absolute minimum time calculated by Dijkstra's algorithm. 3. *Constraint Violation Rate:* A hard count of physically impossible actions proposed by the LLM ($V = \sum c_{invalid}$), such as exceeding vehicle capacities or traversing blocked edges.

Aggregation and Reporting: LLM outputs exhibit non-determinism. To control for this, the evaluation protocol mandates that the agent's temperature parameter is set to 0.0. Each of the 20 tasks must be run 3 times per agent. The final reported results must include the average PWRS, the average Efficiency score, and the total Constraint Violation count across all 60 execution traces.

5 Benchmark Validation

To ensure the benchmark is well-designed and logically sound, we utilized a multi-layered validation approach.

Task Quality and Solvability: We enforce task quality programmatically. All 20 JSON scenarios are parsed through a custom Python schema enforcer (`visualize_city.py`) which renders the map using the NetworkX library. This process visually and mathematically verifies that all nodes are connected and that no impossible map geographies exist. Furthermore, we conducted deterministic pilot testing. We ran a greedy heuristic solver utilizing Dijkstra's algorithm against all tasks; every task was verified to have at least one mathematically possible path to a successful resolution.

Preliminary Results: To validate difficulty scaling, we evaluated the benchmark against three baseline approaches:

- **Greedy Heuristic Solver:** Assigns the nearest available units to the highest-severity incidents.
- **Standard Zero-Shot LLM:** A base model generating a one-pass plan without validation or repair loops.
- **Ablated Agent:** A fully-equipped agent stripped of its ability to process dynamic mid-mission updates.

While the Standard Zero-Shot LLM achieved success on Tier 1 tasks, it failed completely on Tier 4 scenarios, consistently hallucinating travel routes across bridges destroyed by dynamic triggers.

Similarly, the Ablated Agent failed to recalculate ETAs when flooded edges blocked primary hospital access. These preliminary baseline results confirm that RescueBench possesses appropriate difficulty variation and successfully isolates the necessity of complex, self-correcting agent architectures.

6 Conclusion

RescueBench provides a rigorously structured, highly reproducible framework for evaluating an LLM agent's capacity for spatial reasoning and dynamic constraint satisfaction. By grounding the evaluation in deterministic Operations Research mechanics—utilizing 20 graph-based JSON environments—RescueBench moves beyond semantic text evaluation to test adherence to strict physical laws. We anticipate this benchmark will significantly benefit the research community by providing an objective standard for evaluating autonomous planning, self-correction loops, and dynamic adaptability in resource-scarce environments. Future iterations of this benchmark will aim to integrate real-world topological data, bridging the gap between synthetic graphs and physical deployment.

References

K. Li et al. Multi-objective infeasibility diagnosis for routing problems using large language models. *arXiv preprint arXiv:2508.03406*, 2025. URL `https://arxiv.org/abs/2508.03406`.

P. Song et al. Large language model reasoning failures. *arXiv preprint arXiv:2602.06176*, 2026. URL `https://arxiv.org/abs/2602.06176`.

Y. Wang et al. Large language models in operations research: Methods, applications, and challenges. *arXiv preprint arXiv:2509.18180*, 2025. URL `https://arxiv.org/abs/2509.18180`.

S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL `https://arxiv.org/abs/2210.03629`.

N. Zhang et al. An agentic framework with LLMs for solving complex vehicle routing problems. *arXiv preprint arXiv:2510.16701*, 2025. URL `https://arxiv.org/abs/2510.16701`.

A Appendix

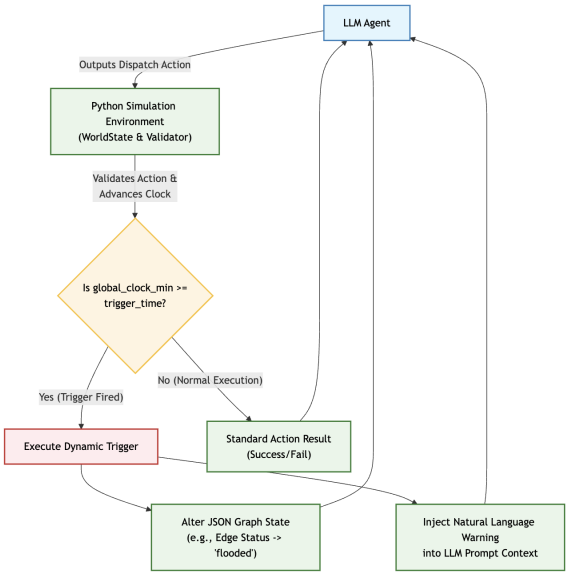

Figure 2: The RescueBench Dynamic Replanning injection loop (Tier 4 scenarios).